Lab 2 SimpleTun Implementation

Instructions to run:

1. Start both VMs using:
    a. VBoxManage startvm CS528_vm1 --type headless
    b. VBoxManage startvm cs528_vm2 --type headless (case sensitive cs528_vm2)
2. SSH into the machines
    a. ssh -p 11259 cs528user@localhost (other is 11260)
3. Run the VPN:
    a. cd to minivpn(symlink) or cd /Desktop/cs528
    b. gcc -o simpletun simpletun.c -lssl -lcrypt on both machines
    c. No changes in the default execution command.
4. Setup the tunnels:
    a. sudo ./config_vm# (replace # by 1 or 2 as you need.)
5. Play around, have a good day!

Note: I have used mc20.cs.purdue.edu as VBoxManage was unusable in mc19.

Questions on page 2

Setup on page 3 – 5

Results from page 6.

Questions:

Question 2.1: Think about why it is better to use UDP in the tunnel, instead of TCP write your answer in your lab report.

Answer:

UDP is faster than TCP.

Due to the following factors: lower overhead, faster transmission speed, connectionless nature facilitating NAT traversal, fault tolerance, support for multiplexing, and better performance over lossy networks.

Question 2.2: Think why it is not recommended to implement your own algorithm; and write that in your lab report.

Answer:

Since we implement only based on our experience, it might not be prone to errors. It doesn't mean that, if we cannot break the algorithm, someone can't.

Question 2.3: Why is it important for the server to release resources when a connection is broken?

Answer:

a. If the server does not release resources, it might take up resources, eventually building up and making the server unusable.
b. For memory resources, memory leaks may occur making the server prone to memory exploitation.
c. For network resources, the connection pool may run out and the server will be unable to create/accept new connections.

Snips and explanation follows… (page 6)

Task 0: Setting up

Importing two VMs:

VBoxManage import /homes/cs528/vms/cs528_vm.ova --vsys 0 --vmname CS528_vm1 --unit 9 --disk /scratch/alluri2/lab1/CS528_vm1/CS528_vm1.vmdk

VBoxManage import /homes/cs528/vms/cs528_vm.ova --vsys 0 --vmname cs528_vm2 --unit 9 --disk /scratch/alluri2/lab1/cs528_vm2/cs528_vm2.vmdk

Note: For some reason, I couldn't create CS528_vm2, so instead, I created cs528_vm2

Also, on mc19.cs.purdue.edu, VBoxManage is stuck. So, I'm using mc20.cs.purdue.edu machine instead.

Register them:

VBoxManage registervm /homes/alluri2/VirtualBox\ VMs/CS528_vm1/CS528_vm1.vbox

VBoxManage registervm /homes/alluri2/VirtualBox\ VMs/cs528_vm2/cs528_vm2.vbox

Setup the network:

I had a nat server setup with 192.168.1.0 and alluri2 as the name, so I went ahead and deleted it from the machine and added the following:

VBoxManage natnetwork add --netname alluri2 --network "192.168.15.0/24" --enable --dhcp on

After starting the natnetwork, the dhcp server showed up.

Map the natnetwork to the vm:

VBoxManage modifyvm CS528_vm1 --nat-network1 alluri2

VBoxManage modifyvm cs528_vm2 --nat-network1 alluri2

Started the **both vms** in headless mode:

VBoxManage startvm CS528_vm1 --type headless

VBoxManage startvm cs528_vm2 --type headless

Got the information using:

VBoxManage guestproperty enumerate CS528_vm1 and for 2.

Setup a port forwarding rule for ssh:

Find the ports: netstat -tuln

Select the open unused ports.

VBoxManage natnetwork modify --netname $USERID --port-forward-4 "guestssh1:tcp:[]:11259:[192.168.15.4]:22"

VBoxManage natnetwork modify --netname $USERID --port-forward-4 "guestssh2:tcp:[]:11260:[192.168.15.5]:22"

Test the ports:

ssh -p 11259 cs528user@localhost

ssh -p 11260 cs528user@localhost

Password: cs528pass for both the machines.

Clean up Shutdown the VM after use:

VBoxManage controlvm CS528_vm1 poweroff

VBoxManage controlvm cs528_vm2 poweroff

SimpleTun (Use tmux for better UI):

Scp'ed simpletun.tar.bz2 from backreference_link, into desktop/simpleton.

sudo ./simpletun -i tun0 -s -d on both machines doesn't work

sudo ./simpletun -i tun0 -c 192.168.15.4 -d on client

On Machine 1 (CS528_vm1) [192.168.15.4]:

sudo ip addr add 10.0.1.1/24 dev tun0

sudo ifconfig tun0 up

sudo route add -net 10.0.2.0 netmask 255.255.255.0 dev tun0

On Machine 2 (cs528_vm2) [192.168.15.5]:

sudo ./simpletun -i tun0 -c 192.168.15.4 -d

sudo ip addr add 10.0.2.1/24 dev tun0

sudo ifconfig tun0 up

sudo route add -net 10.0.1.0 netmask 255.255.255.0 dev tun0

Testing the tunnel:

Ping 10.0.2.1 from machine 1.

Play with ssh: ssh 10.0.2.1 on VM1, then inside the ssh_VM1(VM2): ssh 10.0.1.1 back to VM1 – works.

While doing this, I tried pinging other addresses in 10.0.1.0 subnet, the traffic is being registered on the log of simpletun on both executions, it's rejecting the ping request.

Developing UDP

Tools: WinSCP syncs the src code into both the VMs from windows; watch -n 10 make & #Compiles every 10 seconds.

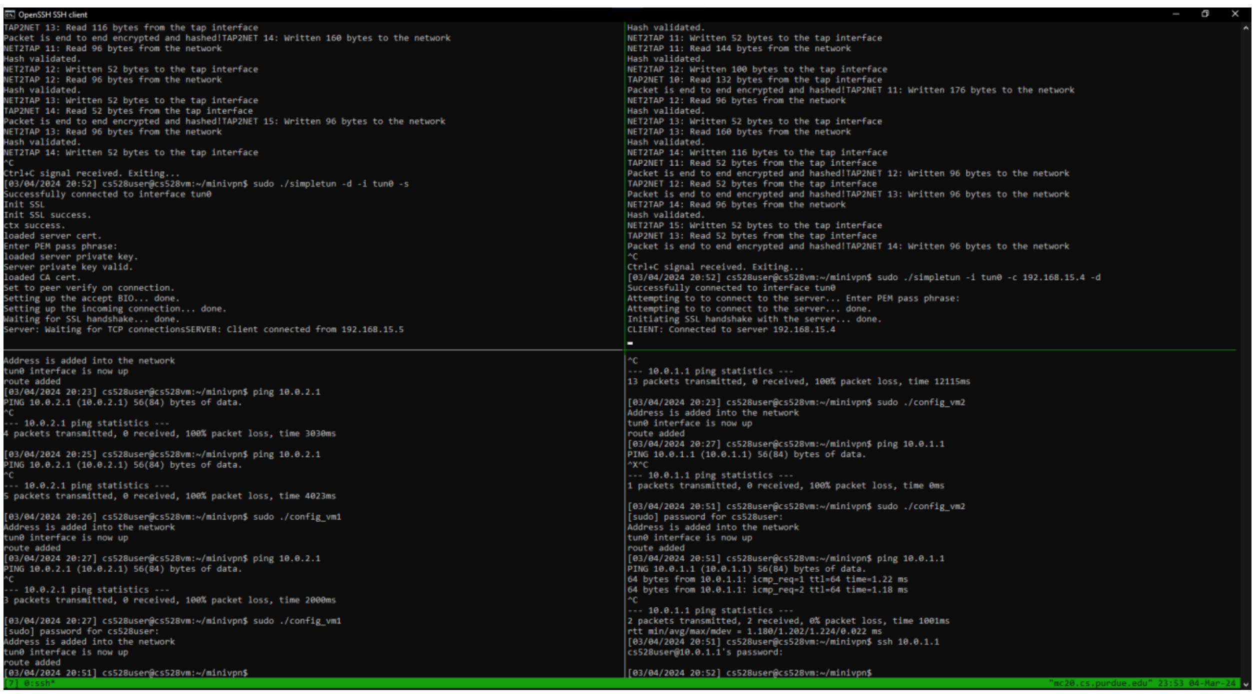
Password for everything: cs528pass

Get the remote ip (client ip) from the TCP handshake that already exists in simpletun.c

Establish a UDP socket using the addresses and bind the local port 55554, since TCP is defaulted to 55555.

For creating SSL certs, I've used the lab 2 instructions.

SSL verification: **(I'm only using the TCP connection to get the client IP address in the server)**

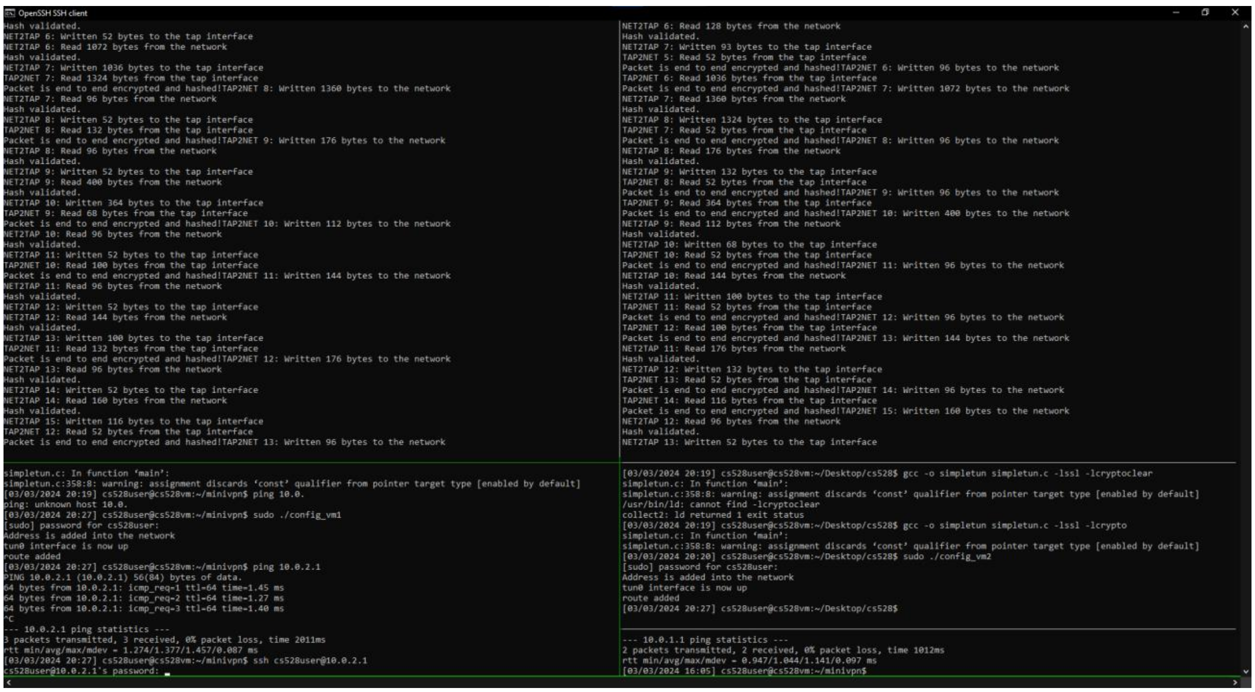


Finished phew!!! (UDP took 3 days. Didn't take a screenshot of just the UDP)

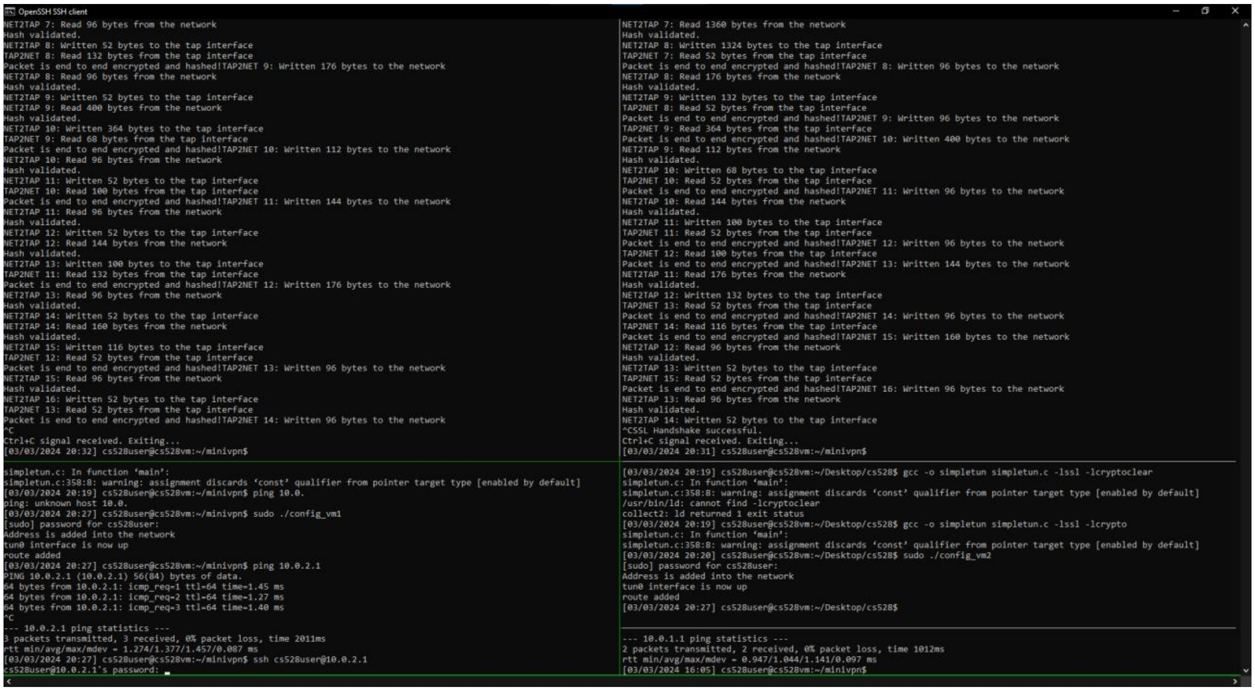
- I've implemented AES CBC encryption for end-to-end encryption.
- HMAC for validation.
- The key and iv are generated based on the SSL session.

I've tested the AES encryption by disabling the encryption, the communication fails. Similar for the HMAC as well.

Final result, ssh works:



Closing and cleaning:



OpenSSL certs setup:

```
Organization Name (eg, company) [Internet Widgits Pty Ltd]:PU
Organizational Unit Name (eg, section) []:PU
Common Name (e.g. server FQDN or YOUR name) []:svr_csr
Email Address []:svr_csr@mc20.cs.purdue.edu

Please enter the following 'extra' attributes
to be sent with your certificate request
A challenge password []:cs528pass
An optional company name []:PU
mc20 113 $ openssl ca -in server.csr -out server.crt -cert ca.crt -keyfile ca.key -config openssl.cnf
Using configuration from openssl.cnf
Enter pass phrase for ca.key:
ca: ./demoCA/newcerts is not a directory
./demoCA/newcerts: No such file or directory
mc20 114 $ cd ..
mc20 115 $ openssl ca -in ./demoCA/server.csr -out ./demoCA/server.crt -cert ./demoCA/ca.crt -keyfile ./demoCA/ca.key
-config ./demoCA/openssl.cnf
Using configuration from ./demoCA/openssl.cnf
Enter pass phrase for ./demoCA/ca.key:
Check that the request matches the signature
Signature ok
Certificate Details:
        Serial Number: 4096 (0x1000)
        Validity
            Not Before: Mar  3 21:28:13 2024 GMT
            Not After : Mar  3 21:28:13 2025 GMT
        Subject:
            countryName               = US
            stateOrProvinceName       = Indiana
            organizationName          = PU
            organizationalUnitName    = PU
            commonName                = svr_csr
            emailAddress              = svr_csr@mc20.cs.purdue.edu
        X509v3 extensions:
            X509v3 Basic Constraints:
                CA:FALSE
            X509v3 Subject Key Identifier:
                FE:CB:30:21:B7:0D:FF:51:4C:59:4B:9B:0D:88:A0:EC:73:BA:22:C7
            X509v3 Authority Key Identifier:
                1F:5C:F1:CE:20:2C:90:44:A8:09:10:BD:2D:82:D8:12:EE:4A:47:90
Certificate is to be certified until Mar  3 21:28:13 2025 GMT (365 days)
Sign the certificate? [y/n]:y


1 out of 1 certificate requests certified, commit? [y/n]y
Write out database with 1 new entries
Data Base Updated
mc20 116 $
```